Algorithms

MINIMUM SPANNING TREES (PART 1)

SOME SLIDES ARE TAKEN FROM ALGORITHMS COURSE, TECHNION

Last Week

- Greedy algorithms
- Huffman Coding

This Week

- Minimum Spanning Trees (MST)
  - Generic rules (Red rule, Blue rule)
  - Kruskal Algorithm

Minimum Spanning Tree Problem

Input: Undirected connected graph G = (V,E) and a cost function

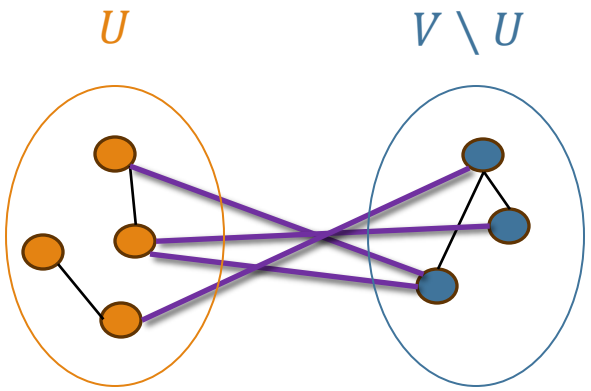
$c: E \to R_{\geq 0}$. $c(e)$ is the cost of edge e.

Output: A spanning tree T with minimum total cost. That is: T that minimizes

$$c(T) = \sum_{e \in T} c(e)$$

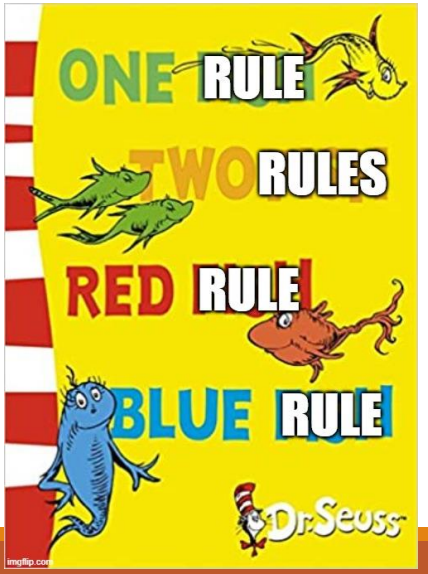
Generic MST Algorithm

- A **cut** in a graph is a partition of the vertices into two disjoint subsets U and $V \setminus U$.
- For a given cut, the **cut-edges** are the edges that have one endpoint in each subset of the cut.

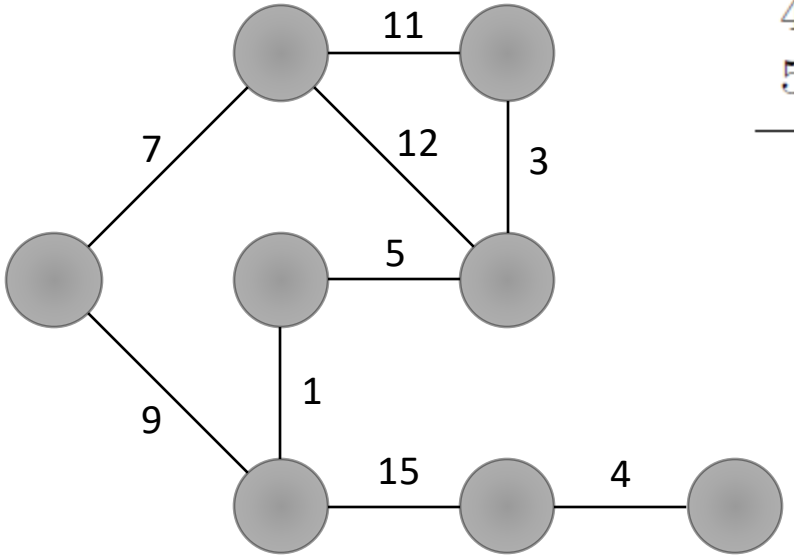


Generic MST Algorithm

1. Blue Rule: Let U be a cut with no blue edges. Select an uncolored edge in U of minimum weight and color it blue.
2. Red Rule: Let C be a cycle with no red edges. Select an uncolored edge of C of maximum weight and color it red.



Kruskal Algorithm

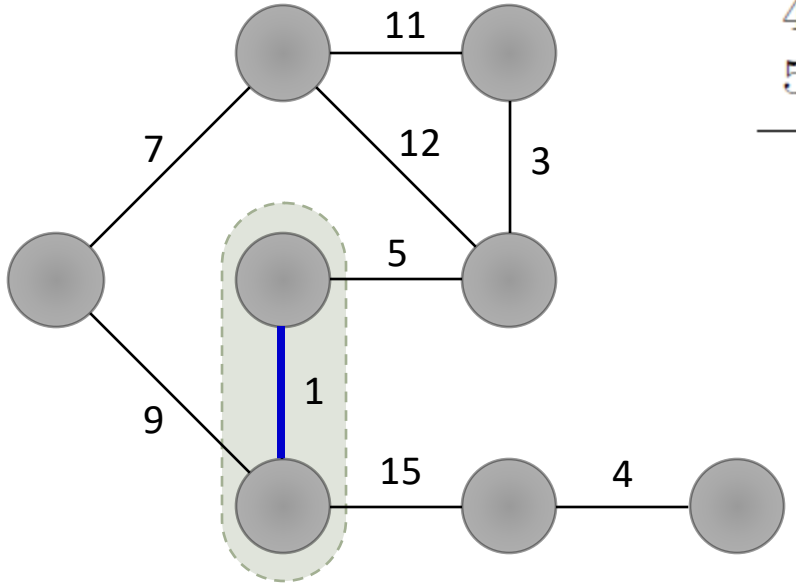


Algorithm 1 KRUSKAL(G, w)

1: $A \leftarrow E$ sorted by $w(e)$ in non-decreasing order
2: $T \leftarrow \{\}$
3: **for** each edge $e \in A$ in increasing order **do**
4: **if** adding e does not form a cycle in T **then**
5: $T = T \cup \{e\}$

Kruskal Algorithm

1, 3, 4, 5, ,7, 9, 11, 12, 15

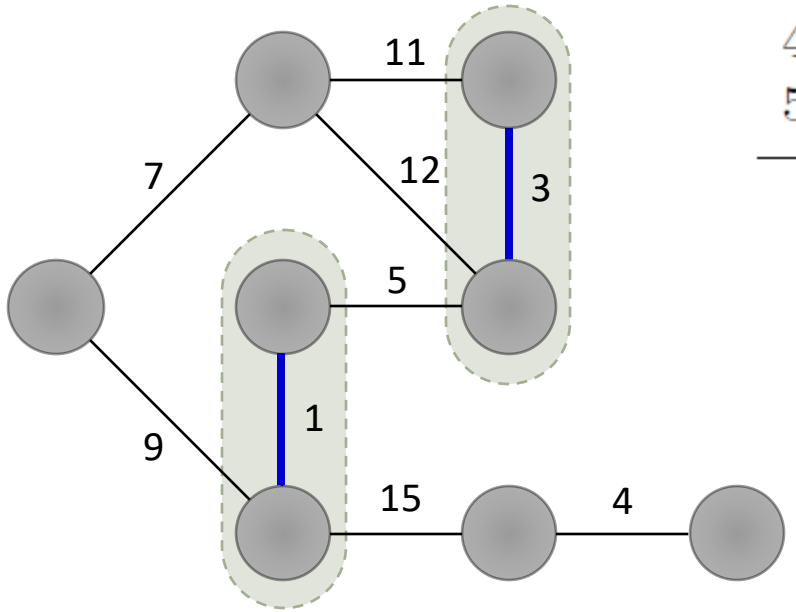


Algorithm 1 KRUSKAL(G, w)

1: $A \leftarrow E$ sorted by $w(e)$ in non-decreasing order
2: $T \leftarrow \{\}$
3: **for** each edge $e \in A$ in increasing order **do**
4: **if** adding e does not form a cycle in T **then**
5: $T = T \cup \{e\}$

Kruskal Algorithm

1, 3, 4, 5, ,7, 9, 11, 12, 15

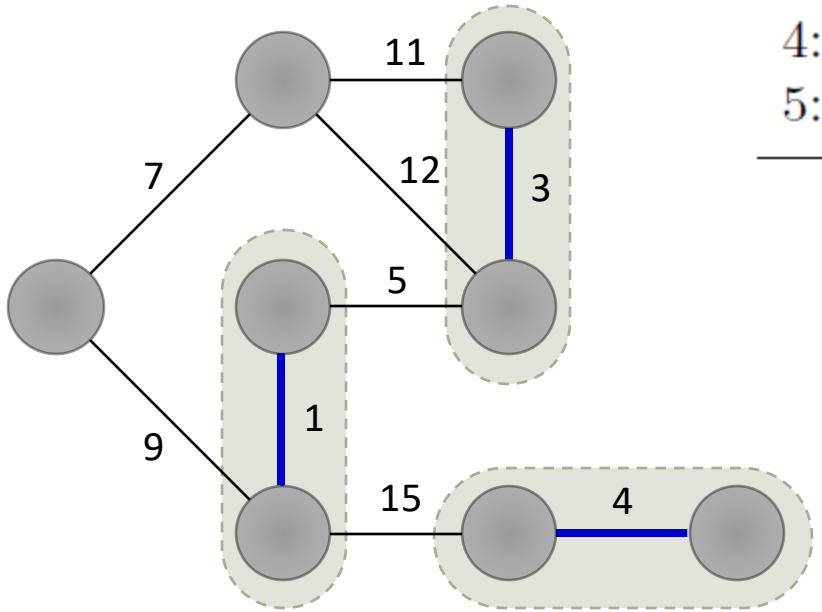


Algorithm 1 KRUSKAL(G, w)

1: $A \leftarrow E$ sorted by $w(e)$ in non-decreasing order
2: $T \leftarrow \{\}$
3: **for** each edge $e \in A$ in increasing order **do**
4: **if** adding e does not form a cycle in T **then**
5: $T = T \cup \{e\}$

Kruskal Algorithm

1, 3, 4, 5, ,7, 9, 11, 12, 15

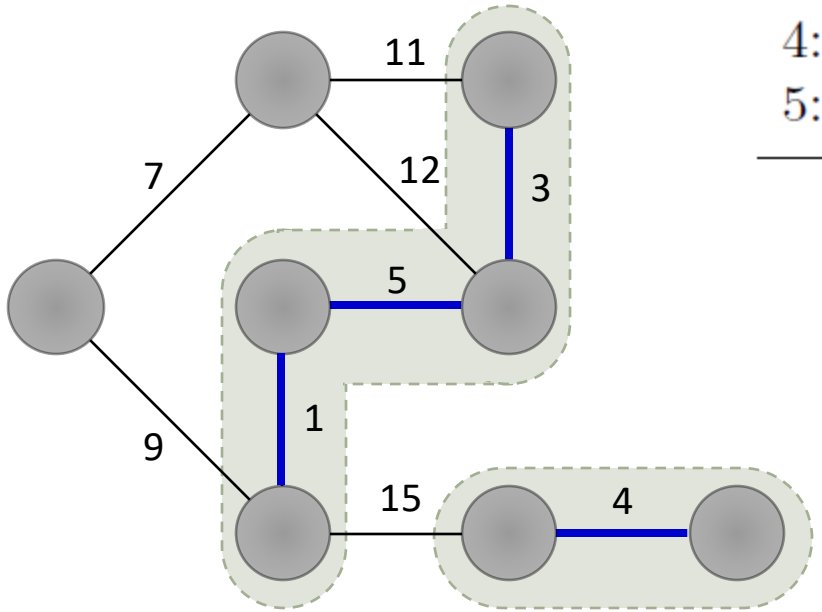


Algorithm 1 KRUSKAL(G, w)

1: $A \leftarrow E$ sorted by $w(e)$ in non-decreasing order
2: $T \leftarrow \{\}$
3: **for** each edge $e \in A$ in increasing order **do**
4: **if** adding e does not form a cycle in T **then**
5: $T = T \cup \{e\}$

Kruskal Algorithm

1, 3, 4, 5, ,7, 9, 11, 12, 15

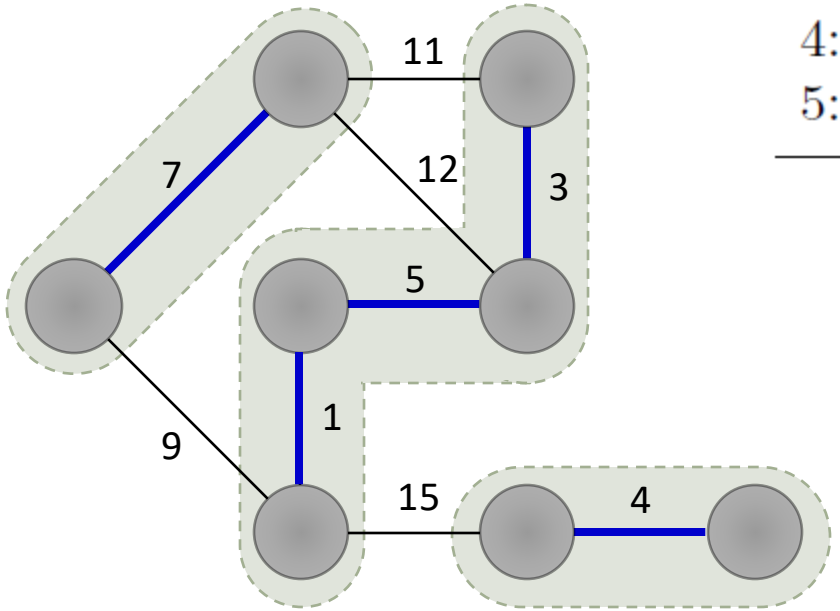


Algorithm 1 KRUSKAL(G, w)

1: $A \leftarrow E$ sorted by $w(e)$ in non-decreasing order
2: $T \leftarrow \{\}$
3: **for** each edge $e \in A$ in increasing order **do**
4: **if** adding e does not form a cycle in T **then**
5: $T = T \cup \{e\}$

Kruskal Algorithm

1, 3, 4, 5, ,7, 9, 11, 12, 15

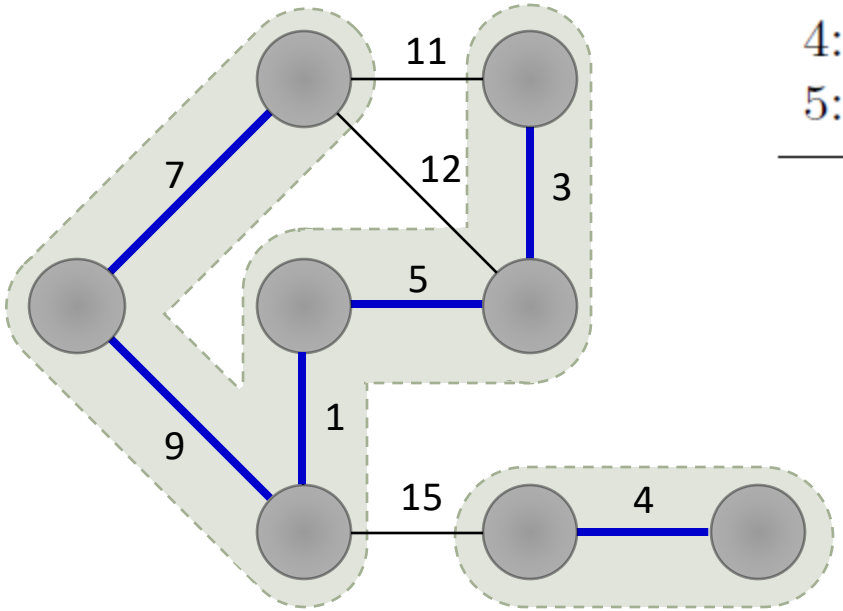


Algorithm 1 KRUSKAL(G, w)

1: $A \leftarrow E$ sorted by $w(e)$ in non-decreasing order
2: $T \leftarrow \{\}$
3: **for** each edge $e \in A$ in increasing order **do**
4: **if** adding e does not form a cycle in T **then**
5: $T = T \cup \{e\}$

Kruskal Algorithm

1, 3, 4, 5, ,7, 9, 11, 12, 15

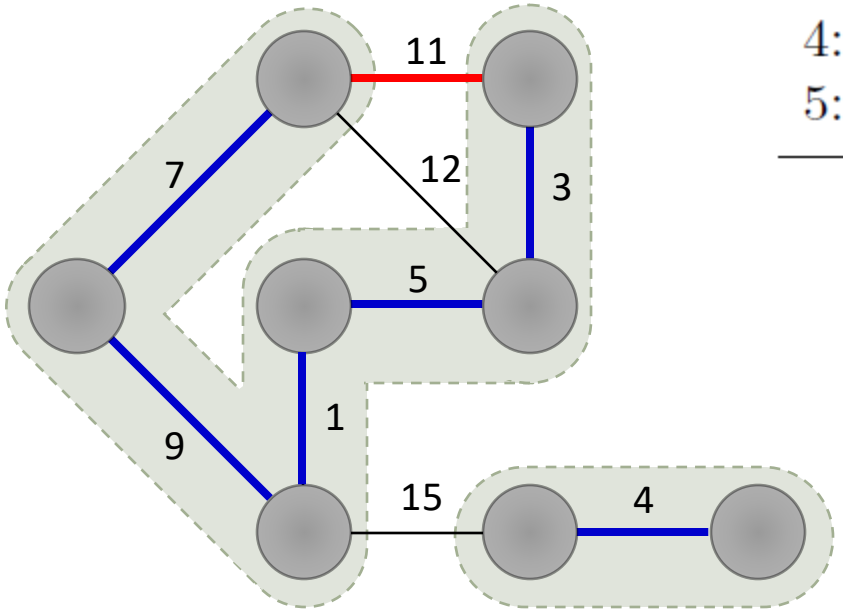


Algorithm 1 KRUSKAL(G, w)

1: $A \leftarrow E$ sorted by $w(e)$ in non-decreasing order
2: $T \leftarrow \{\}$
3: **for** each edge $e \in A$ in increasing order **do**
4: **if** adding e does not form a cycle in T **then**
5: $T = T \cup \{e\}$

Kruskal Algorithm

1, 3, 4, 5, ,7, 9, 11, 12, 15

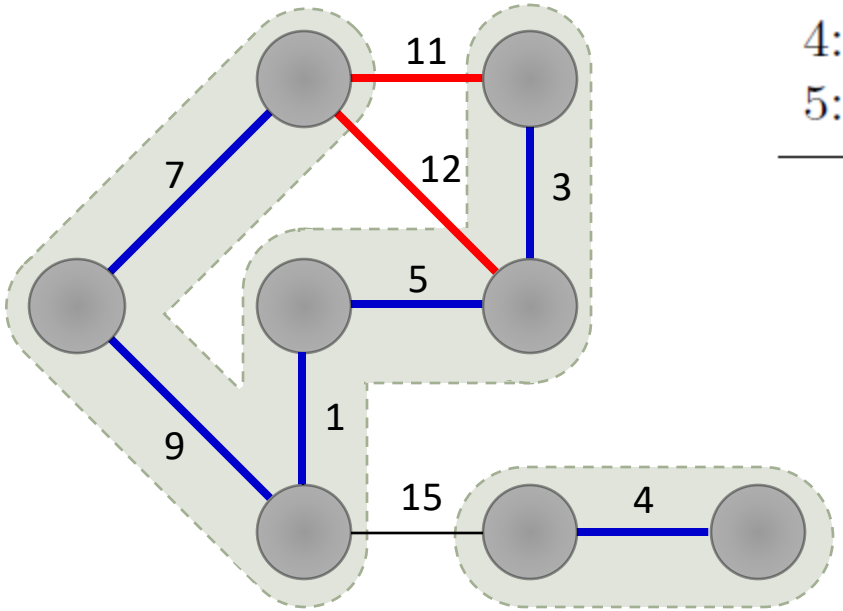


Algorithm 1 KRUSKAL(G, w)

1: $A \leftarrow E$ sorted by $w(e)$ in non-decreasing order
2: $T \leftarrow \{\}$
3: **for** each edge $e \in A$ in increasing order **do**
4: **if** adding e does not form a cycle in T **then**
5: $T = T \cup \{e\}$

Kruskal Algorithm

1, 3, 4, 5, ,7, 9, 11, 12, 15

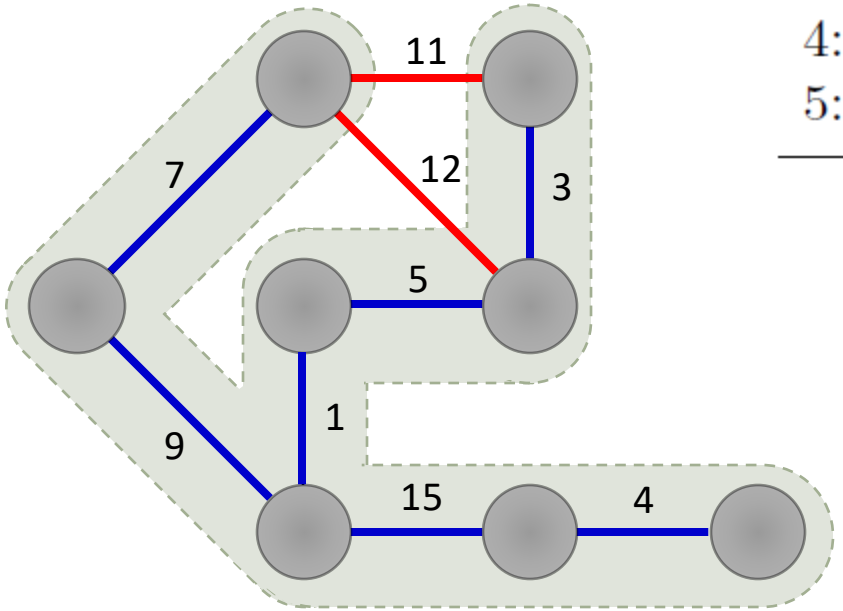


Algorithm 1 KRUSKAL(G, w)

1: $A \leftarrow E$ sorted by $w(e)$ in non-decreasing order
2: $T \leftarrow \{\}$
3: **for** each edge $e \in A$ in increasing order **do**
4: **if** adding e does not form a cycle in T **then**
5: $T = T \cup \{e\}$

Kruskal Algorithm

1, 3, 4, 5, ,7, 9, 11, 12, 15



Algorithm 1 KRUSKAL(G, w)

1: $A \leftarrow E$ sorted by $w(e)$ in non-decreasing order
2: $T \leftarrow \{\}$
3: **for** each edge $e \in A$ in increasing order **do**
4: **if** adding e does not form a cycle in T **then**
5: $T = T \cup \{e\}$

Kruskal Algorithm Implementation

Data Structure: Disjoint-set

Supports:

- Find(a)
- Union(a,b)

Algorithm 1 KRUSKAL(G, w)

1: $A \leftarrow E$ sorted by $w(e)$ in non-decreasing order
2: $T \leftarrow \{\}$
3: **for** each edge $e \in A$ in increasing order **do**
4: **if** adding e does not form a cycle in T **then**
5: $T = T \cup \{e\}$

- Step 4 can be done with at most 2 find operations and one Union. Why? Because T is a forest.

Kruskal Algorithm

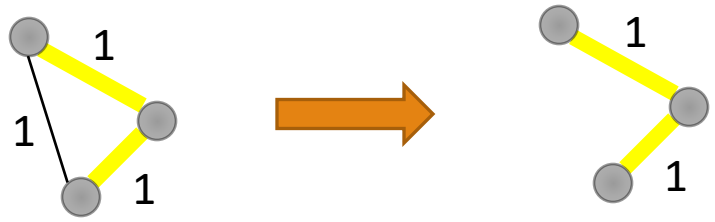
Time Complexity:

- Sorting the edges takes $O(mlogm)$.
- For each edge, e:
  - Check if adding e to T forms a cycles:
    - $O(m) * [2\ find + union]$.
    - Naïve Disjoint set implementation:$O(mlogn)$.
    - Possible: $O(\alpha m)$.
  - Total time: $O(mlogm)$.

Exercise 1

Let G = (V, E) be an undirected weighted graph, and assume some edges are yellow. Suggest an algorithm to find the most yellow MST. Meaning, find a MST with the maximal number of yellow edges among all MSTs.

For example, this graph has 3 MSTs. The algorithm should return the most yellow one:



Exercise 1 - Solution

Run Kruskal algorithm, where in the sorting stage break ties in favor of yellow edges.

Exercise 1 - Solution

Correctness:

Denote by $T^{ALG} = (V, E^{ALG} \subseteq E)$ the tree returned by the algorithm. From the correctness of Kruskal, T^{ALG} is a MST.

Need to prove: this is the most yellow MST.

Assume towards contradiction that a more yellow MST exists, and let $T^{yel} = (V, E^{yel} \subseteq E)$ be a more yellow MST which is closest to T^{ALG}, meaning $|E^{ALG} \cap E^{yel}|$ is maximal.

Exercise 1 - Solution

Correctness:

Let $e^{yel} = (a, b) \in E^{yel} \backslash E^{ALG}$ be a yellow edge in T^{yel} that doesn't appear in T^{ALG} (there is such an edge since T^{yel} is more yellow).

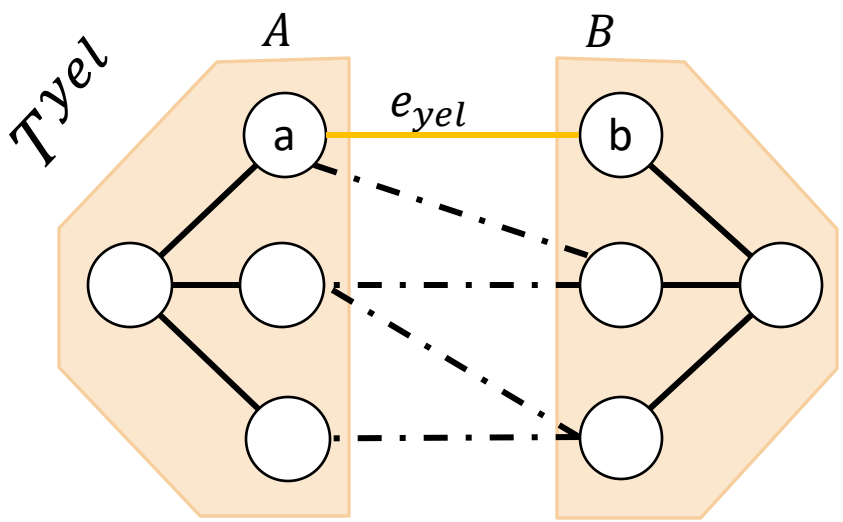
Denote by A, B=V\A the two connected components resulting from the removal of e^{yel} from T^{yel}.

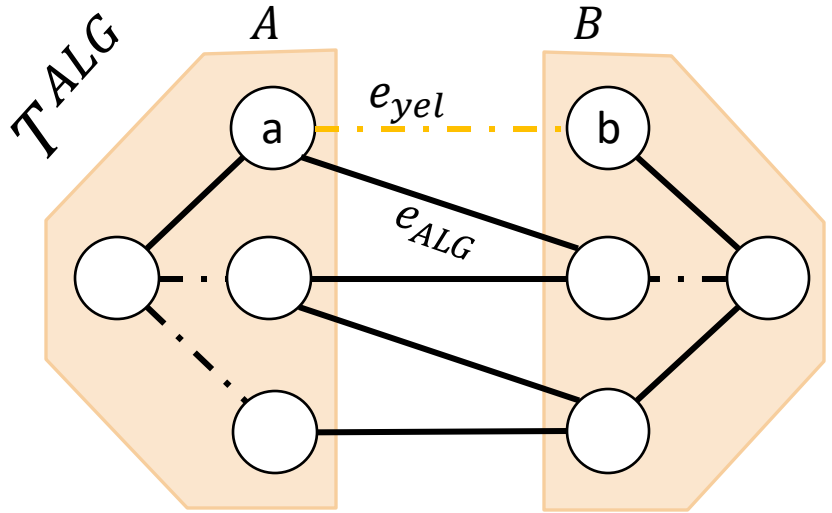
Consider the path p joining the endpoints of e^{yel} in T^{ALG}. Since e^{yel} connects A and B, so does p, and therefore there exists an edge e^{ALG} in p that connects A and B.

Exercise 1 - Solution

Correctness:

Consider the cut (A,B):





Exercise 1 - Solution

Correctness:

Claim 1. The edges e^{ALG} and e^{yel} have the same weight.

Proof.

Note that $E^{yel} \setminus \{e^{yel}\} \cup \{e^{ALG}\}$ form is a ST, and also

$E^{ALG} \setminus \{e^{ALG}\} \cup \{e^{yel}\}$ form a ST. Therefore,

- w(e^{ALG}) > w(e^{yel}) contradicts the fact that T^{ALG} is a MST.
- w(e^{ALG}) < w(e^{yel}) contradicts the fact that T^{yel} is a MST.

■

Exercise 1 - Solution

Correctness:

Claim 2. e^{ALG} is yellow.

Proof. By Claim 1, e^{ALG} and e^{yel} have the same weight. In our algorithm e^{ALG} was chosen before e^{yel}. Since yellow edges have high priority, e^{ALG} must be yellow. ■

Exercise 1 - Solution

Correctness:

Define a new tree $T' = (V, E' = E^{yel} \backslash \{e^{yel}\} \cup \{e^{ALG}\})$ – the tree formed by removing e^{yel} and adding e^{ALG}.

Since both e^{ALG} and e^{yel} are yellow, T' and T^{yel} have the same number of yellow edges.

Claim 3. T' is a MST.

Exercise 1 - Solution

Correctness:

Claim 3. T' is a MST.

Proof. T' was produced by connecting two spanning trees (of A and B). Therefore, it spans the entire graph and has no cycles and $w(e^{ALG}) = w(e^{yel})$. ■

Since both edges are yellow and have the same weight, we have: $|E^{ALG} \cap E^{yel}| < |E^{ALG} \cap E'|$. We get that T' is a MST closer to T^{ALG} than T^{yel}, which is a contradiction to the choice of T^{yel} as the most yellow MST closest to T^{ALG}.

Exercise 1 - Solution

Time Complexity:

The complexity of running Kruskal algorithm - $O(mlogn)$.

Exercise 2

Prove that for any MST of G, there exists a run of Kruskal that finds it.

Exercise 2 - Solution

- Intuition: Let T be some MST. According to previous exercise, if we color the edges of T in yellow, we will get T.
- Proof: Let T be some MST. Define a coloring of the edges in the following way: if $e \in T$ we color the edge yellow, and black otherwise. We run the algorithm for most yellow MST. From exercise 1, we will get T.

Exercise 3

Let G = (V, E) be an undirected connected graph and let $w, w' : E \to \mathbb{R}$ be two weight functions such that:

$$\forall e_1, e_2 \in E : w(e_1) \leq w(e_2) \iff w'(e_1) \leq w'(e_2)$$

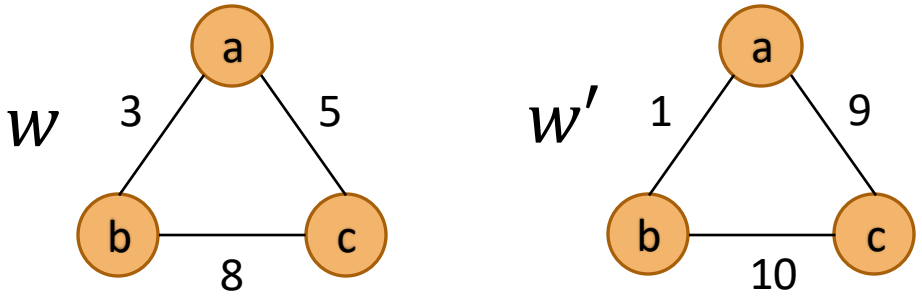
Show that T is a MST w.r.t. w iff T is a MST w.r.t. w'.

For example:

In both weight functions,

$w(ab) \leq w(ac) \leq w(bc)$

$w'(ab) \leq w'(ac) \leq w'(bc)$



Exercise 3 - Solution

$\Longrightarrow$Let T be some MST w.r.t. w. We know that there is some run of Kruskal that returns T. Assume the edge order that returns T is given by $e_1, \ldots, e_m$ and that $w(e_1) \leq \cdots \leq w(e_m)$. This implies $w'(e_1) \leq \cdots \leq w'(e_m)$. Thus, running Kruskal with the weight function w' using this edge order will yield T. Hence, T is a MST w.r.t. w'.

$\Longleftarrow$ Same (replace w with w' and vice versa).

Exercise 4

Let G = (V, E) be an undirected, weighted and connected graph. Show that for every weight x, every MST of G includes the same number of edges of weight x.

Exercise 4 - Solution

Let T_1, T_2 be MSTs of G. Since both are MSTs, $w(T_1) = w(T_2)$.

Denote n_1, n_2 - number of edges with weight x in T_1, T_2 respectively, and assume towards contradiction that $n_1 > n_2$.

Define a new weight function as follows:

$$w'(e) = \begin{cases} x + \epsilon, & w(e) = x \\ w(e), & else \end{cases}$$

(where ϵ is chosen such that the sorted order of the weights doesn't change).

Exercise 4 - Solution

Since w, w’ hold the conditions of exercise 3, we can use its result: both T_1, T_2 are MSTs of G w.r.t. w’ - $w'(T_1) = w'(T_2)$.

However: $w'(T_1) = w(T_1) + n_1\epsilon > w(T_2) + n_2\epsilon = w'(T_2)$

since: $w(T_1) = w(T_2), n_1 > n_2$.

We got both: $w'(T_1) = w'(T_2), w'(T_1) > w'(T_2)$.

This is a contradiction.